

## MODULE 10

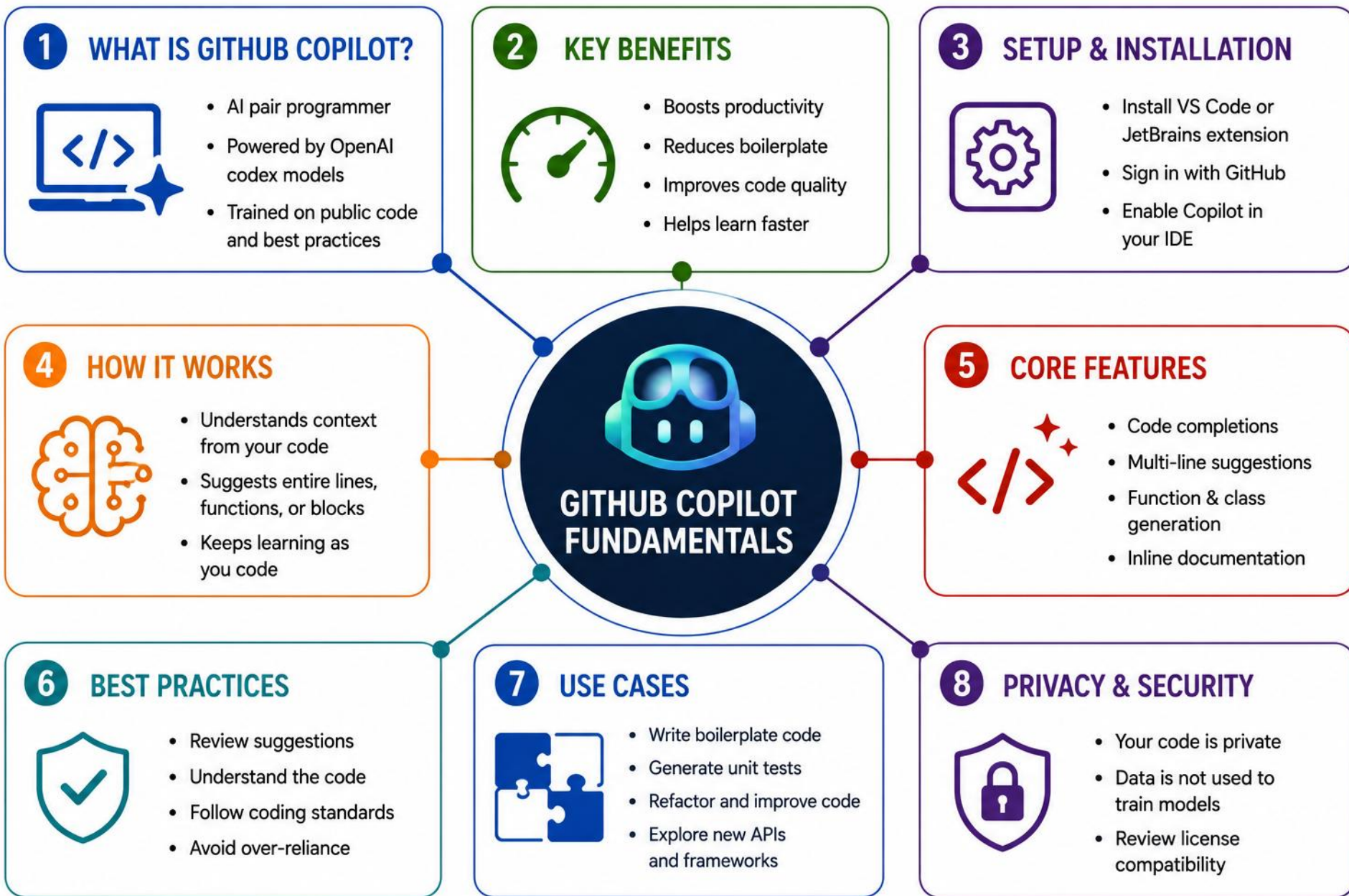
# GitHub Copilot Fundamentals for Java Developers

Use GitHub Copilot to accelerate development, write better code, and improve productivity — responsibly.









# MODULE 10 OVERVIEW

*AI-assisted development is now part of enterprise software engineering — developers must learn when to trust AI, and when to verify it.*

- A developer needs to rapidly scaffold a new customer service. Copilot accelerates development — but every suggestion still needs a human reviewer.

## DURATION & LAB



### 2 Hours Theory

How Copilot works, prompting, generation, review.



### 45 Min Lab

AI-Assisted Code Generation.



### Business Scenario

Scaffold the Northstar CRM customer domain with Copilot.

## COPILOT IS

A context-aware AI pair programmer, not an autopilot

## YOU DECIDE

Every suggestion is reviewed, tested, and refined by you

## THE GOAL

Write better code, faster, without losing engineering judgment

**KEY TAKEAWAY** AI is a force multiplier — it augments your skills, it doesn't replace your judgment.

# LEARNING OBJECTIVES

*By the end of this module, you will use GitHub Copilot effectively to write, explain, refactor, and test code.*

- 1 Understand what GitHub Copilot is and how it works.
- 2 Set up and configure Copilot in your dev environment.
- 3 Generate context-aware code and complete tasks faster.
- 4 Write clear prompts for better, more accurate suggestions.
- 5 Explain and document code with natural language.
- 6 Refactor and improve code using Copilot.
- 7 Generate tests and handle edge cases.
- 8 Review and validate AI-generated code critically.
- 9 Understand risks, limitations, and ethical use.
- 10 Apply Copilot to real enterprise use cases.

## KEY BENEFITS



**Boost Productivity**



**Improve Code Quality**

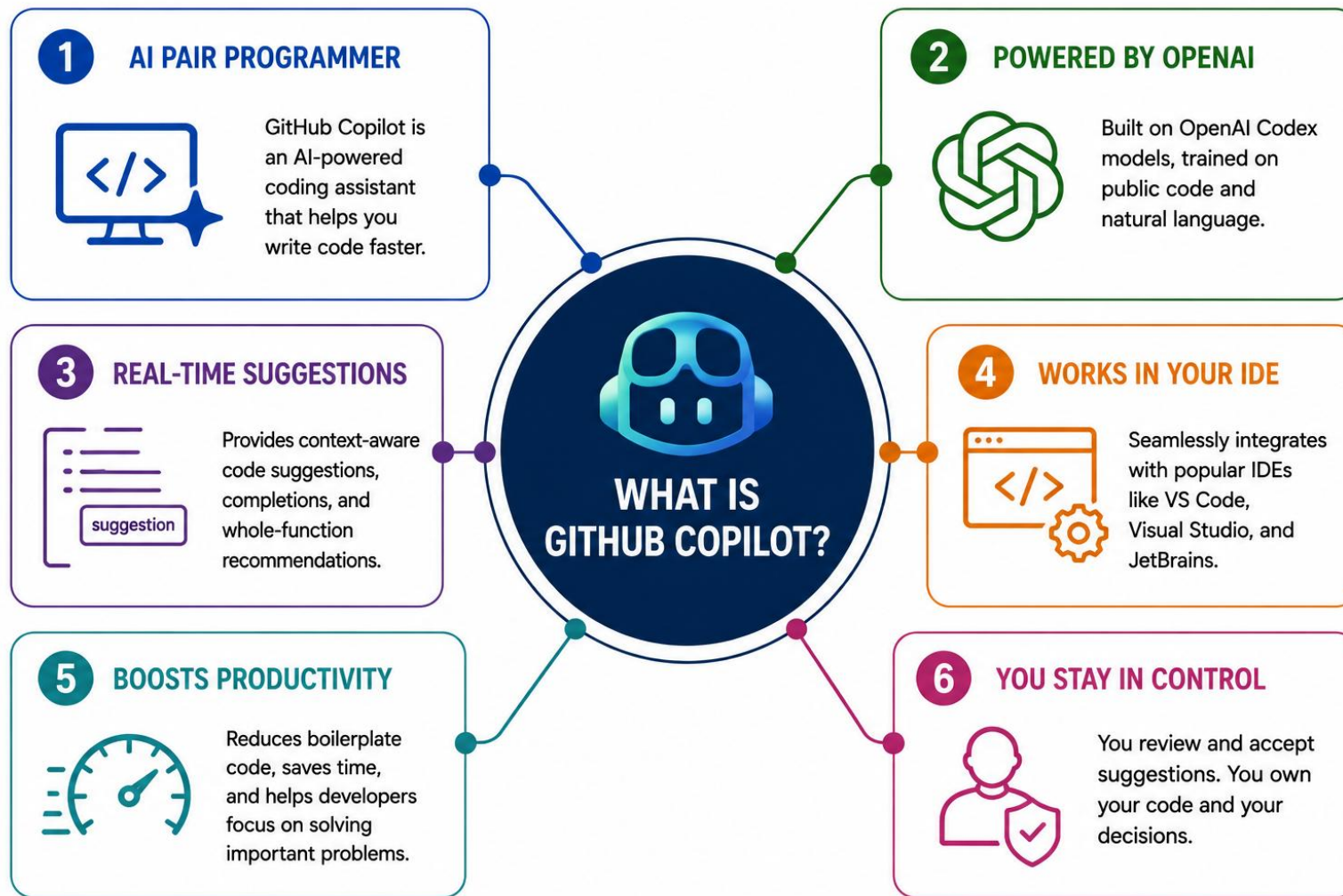


**Accelerate Learning**



**Focus on Impact**

**KEY TAKEAWAY** You will be confident using GitHub Copilot to write better code, move faster, and deliver high-quality software.

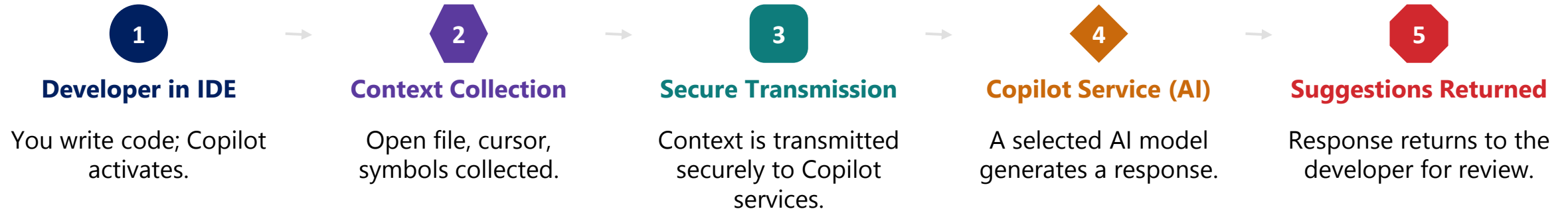


**WHAT IS GITHUB COPILOT** GitHub Copilot is an AI coding assistant that uses editor and repository context to generate suggestions, explanations, and code-related responses.

It uses large language models selected and operated through GitHub's service, and adapts its suggestions to the context available in your editor, files, repository, instructions, and conversation — not by retraining on your code.

# COPILOT ARCHITECTURE OVERVIEW

*Copilot integrates into your IDE, securely sends context to GitHub's AI service, and returns intelligent suggestions in real time.*



## KEY COMPONENTS

- IDE / Editor — extends your IDE, captures context.
- Context Engine — selects relevant code, filters sensitive data.
- Secure Gateway — encrypts and transmits data.
- AI Service — a selected model generates a response.

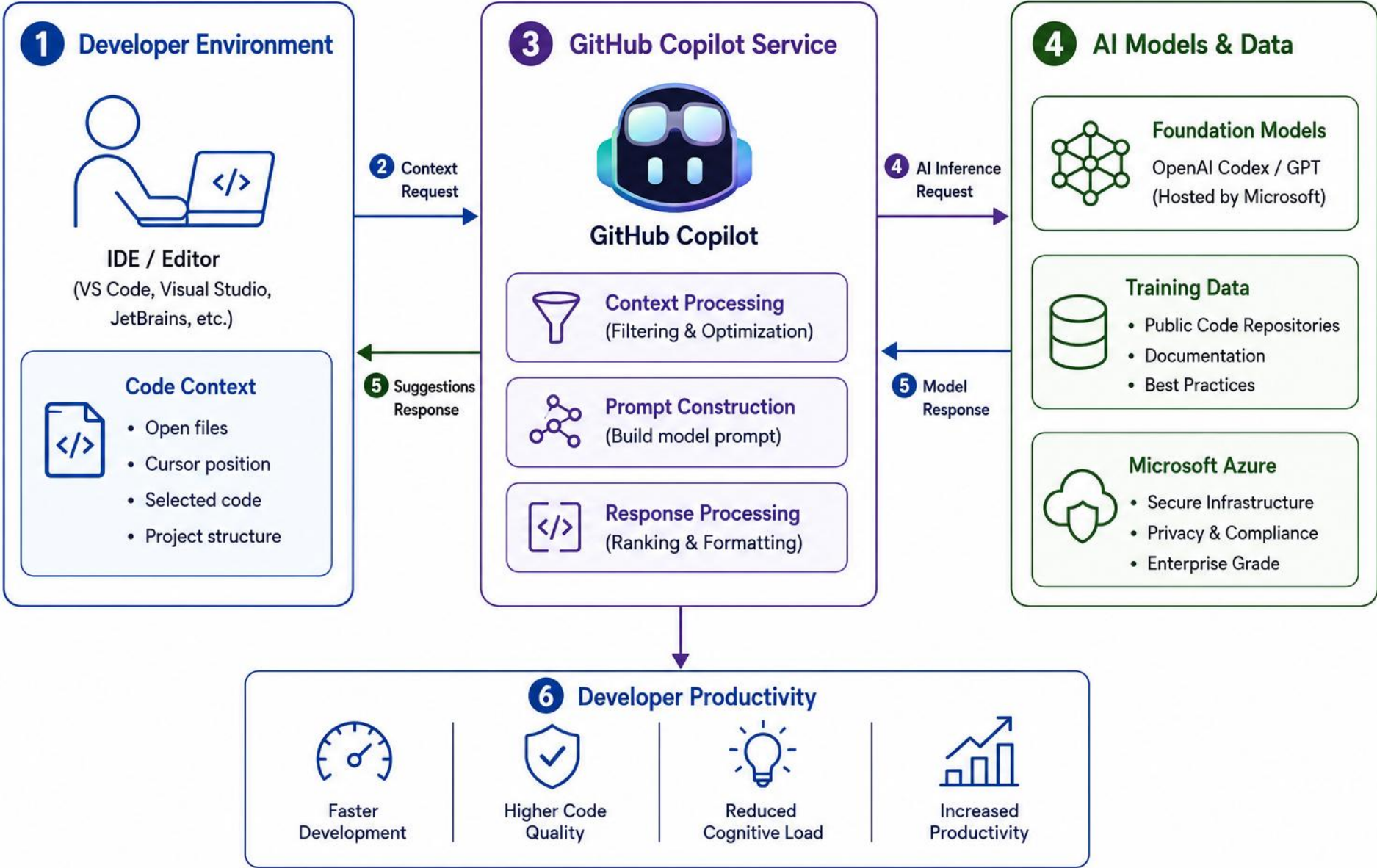
## SECURITY & PRIVACY

- Data handling depends on your Copilot plan, org policy, settings, and current product terms.
- Follow your organization's Copilot policy at all times.
- Never place secrets, credentials, or regulated data in prompts.

**KEY TAKEAWAY** Copilot combines large language models, secure cloud infrastructure, and deep IDE integration to deliver real-time assistance.

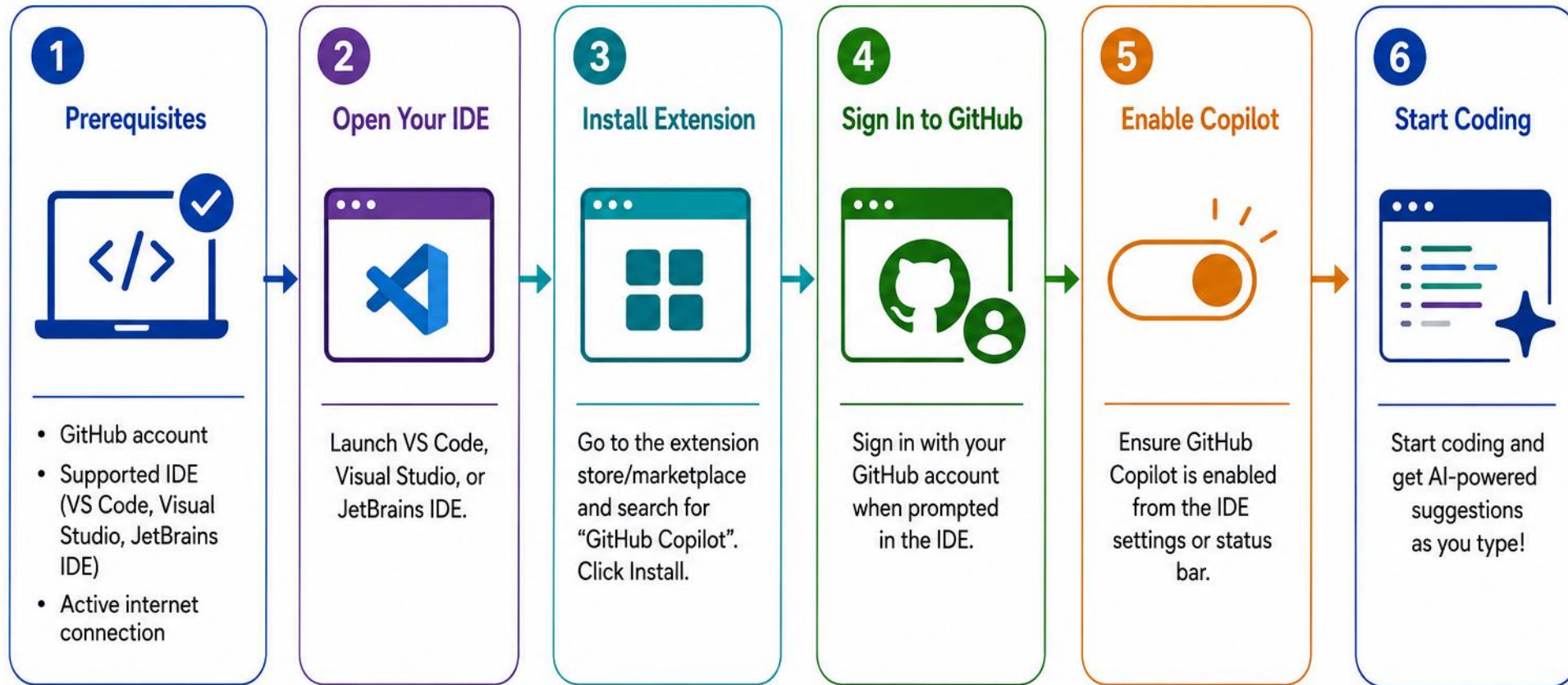


# Copilot Architecture





# Installing GitHub Copilot



**Note:** You need an active GitHub Copilot subscription to use the service.

# COPILOT IN INTELLIJ IDEA

*Copilot integrates seamlessly with IntelliJ IDEA to help you write, understand, and improve code with AI-powered suggestions.*

## SETUP IN 3 STEPS

- Go to File > Settings > Plugins > Marketplace, search 'GitHub Copilot', and install.
- Click the Copilot icon in the toolbar and sign in with your GitHub account.
- Authorize Copilot when prompted, then open a file and start typing.

## GITHUB COPILOT CHAT

```
> Explain this method.
```

```
This method searches for a user in the  
given list whose ID matches the provided  
id, using Java Streams, and returns an  
Optional<User>. Would you like null checks?
```

## TIPS FOR BEST RESULTS



### Write Clear Comments

Guide suggestions with intent.



### Break Into Small Steps

Complex tasks, one step at a time.



### Review Carefully

Before accepting any suggestion.



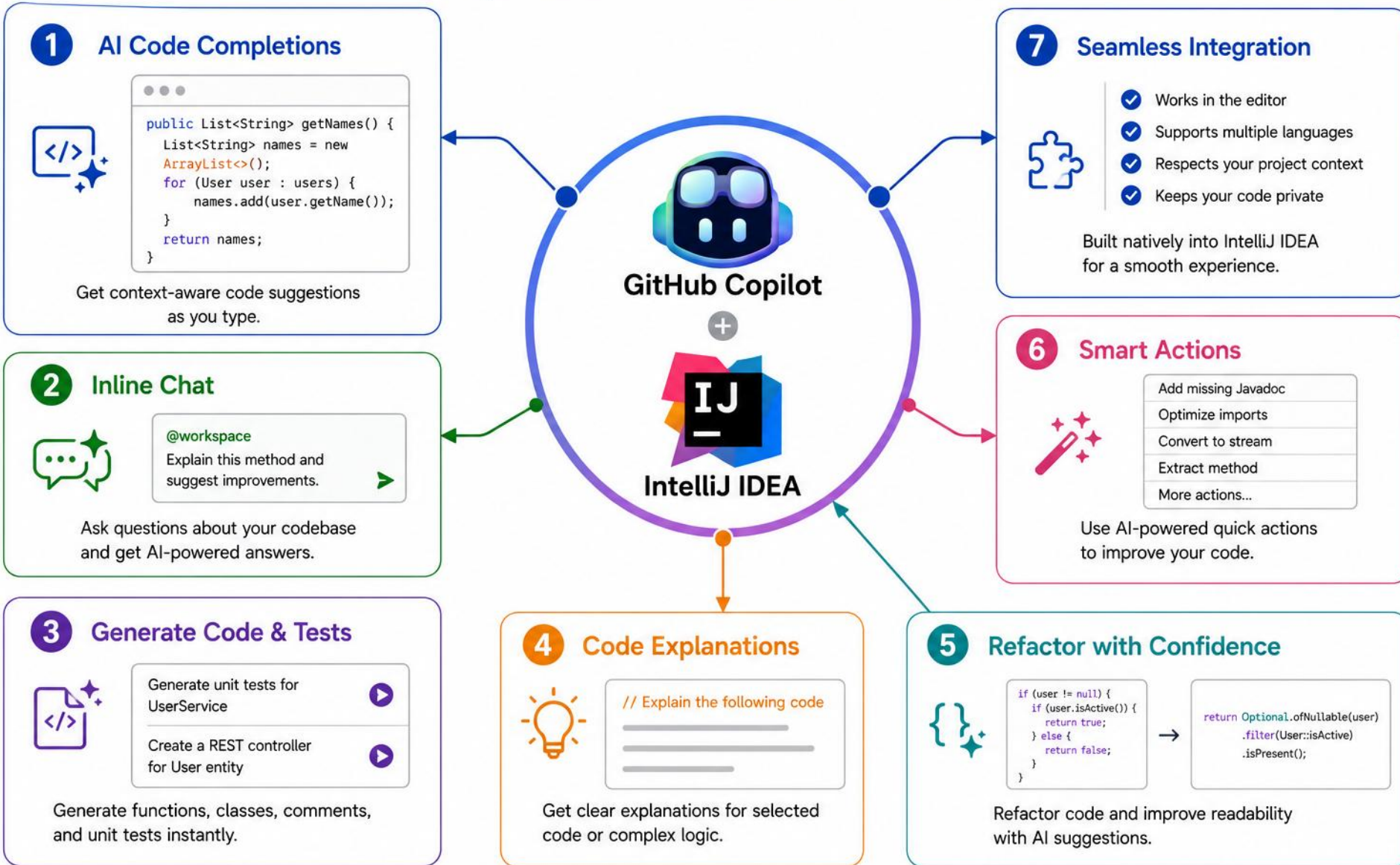
### Use Copilot Chat

Learn, refactor, explore alternatives.

- Keyboard shortcuts: Tab to accept a suggestion, Esc to dismiss, Alt+] / Alt+[ to cycle through alternative suggestions.

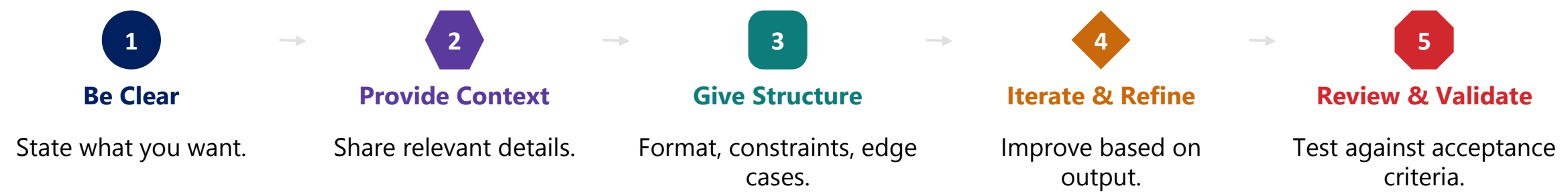
**KEY TAKEAWAY** Copilot in IntelliJ IDEA acts as your AI pair programmer — understanding context, suggesting code, helping you build faster.

# Copilot in IntelliJ IDEA



# PROMPT ENGINEERING FUNDAMENTALS

*A good prompt helps Copilot understand your intent, use the right context, and generate accurate, high-quality code.*



| TASK         | WEAK PROMPT (VAGUE)           | STRONG PROMPT (EFFECTIVE)                                                     |
|--------------|-------------------------------|-------------------------------------------------------------------------------|
| Sorting      | "Sort a list."                | "Sort a List<Employee> by salary, descending, using Comparator."              |
| API Endpoint | "Create an API to get users." | "Create a Spring Boot GET /users/{id} returning JSON, 404 if not found."      |
| Bug Fixing   | "Fix this error."             | "NullPointerException in UserService line 45 — user can be null; fix safely." |

- Prompt improvement workflow: Write Prompt → Get Response → Review Output → Refine Prompt → Better Output (repeat as needed).

**KEY TAKEAWAY** Great prompts = goal + context + constraints + expected output + edge cases + acceptance criteria.



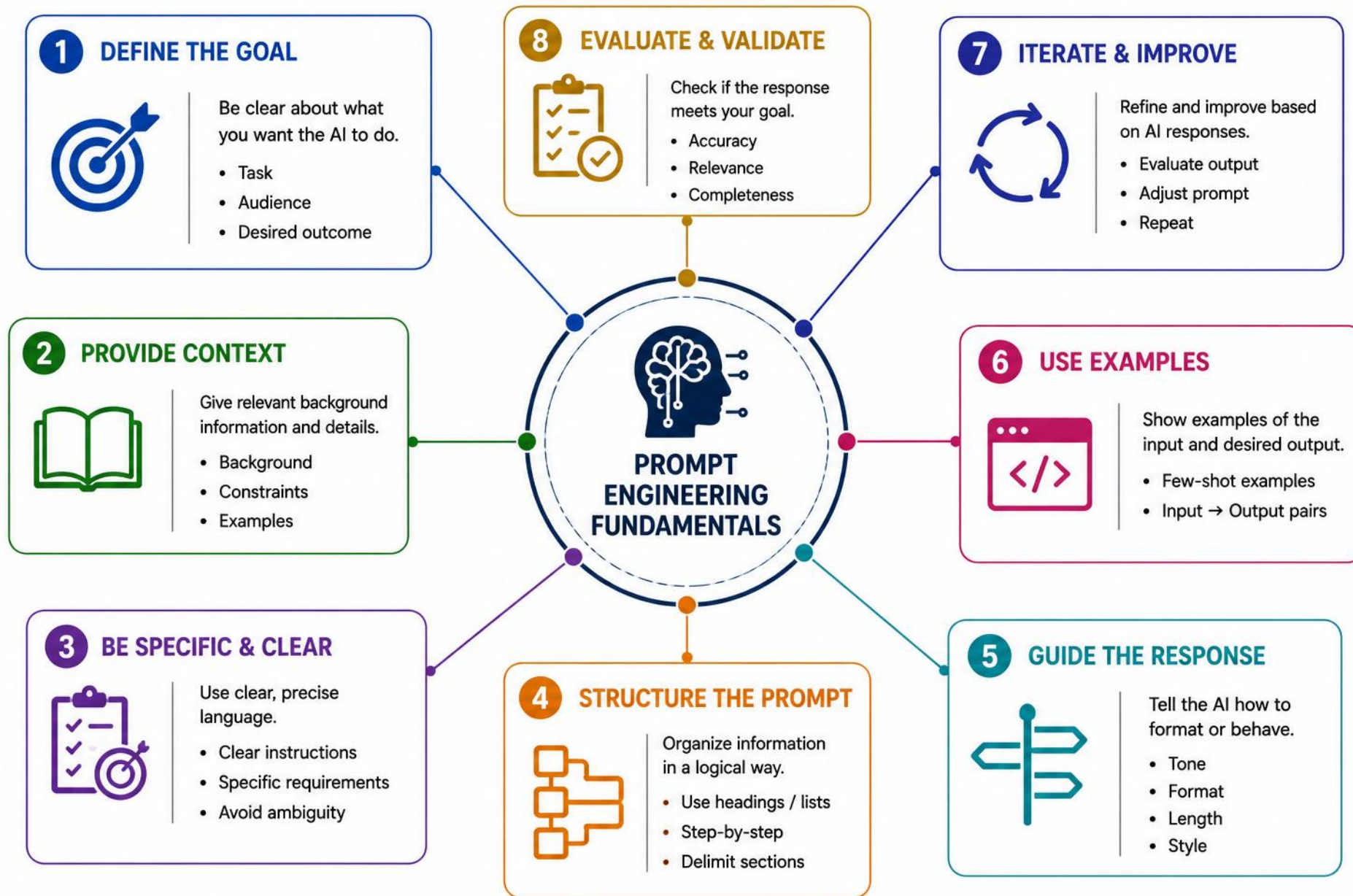
# TRY IT YOURSELF — EXERCISE 1: WEAK VS STRONG PROMPTS

*Reinforces: the weak-prompt vs strong-prompt table you just saw.*

| STEP          | WHAT TO DO                                                                             |
|---------------|----------------------------------------------------------------------------------------|
| Goal          | Contrast a vague Copilot prompt with a strong Northstar-scoped one                     |
| File          | lab10-prelab-prompts.md (in examples/module-10-exercises/notes/)                       |
| Weak prompt   | "Write a customer class." — invites wrong package, JDK APIs, invented annotations      |
| Strong prompt | Java 21 record for CUS-1001 Amina Khan ACTIVE; id, fullName, status; no Spring, no JPA |
| Constraints   | List three: JDK version, domain fixtures, and the no-framework boundary                |
| Reference     | exercises/exercise-01-weak-vs-strong-prompts.md                                        |

```
$ cat notes/lab10-prelab-prompts.md
Weak:  "Write a customer class."  // wrong package, JDK APIs, phantom annotations
Strong: Java 21 record, Northstar CRM,
        CUS-1001 Amina Khan, status ACTIVE  // id, fullName, status only
Constraints: JDK 21, domain fixtures, no Spring/JPA
```

**TRY IT NOW** Complete Exercise 1 before continuing — see exercises/exercise-01-weak-vs-strong-prompts.md.



# GENERATING DOMAIN CLASSES AND DTOS

*Copilot can scaffold a domain class and its DTO from a clear prompt — you review field types, validation, and where the boundary belongs.*

## HOW TO GENERATE A CLASS OR DTO

- Write a clear comment describing the class you want to generate.
- Begin typing the class declaration — Copilot suggests the rest.
- Press Tab (or Enter) to accept the suggested code.
- Review the generated code and refine names, fields, or logic.

## EXAMPLE PROMPT

```
// Create a Java class Customer with customerId  
// (Long), fullName (String), email (String),  
// phone (String), status (CustomerStatus),  
// createdAt (LocalDateTime). Include getters,  
// setters, constructors, and toString().  
public class Customer {
```

# DTO ANNOTATIONS AND BEST PRACTICES

*Once Copilot drafts the class, apply these annotation patterns to validate, build, and serialize it correctly.*

## DTO ANNOTATIONS — WHAT GOES WHERE



### Bean Validation

@NotNull, @NotBlank,  
@Size.



### Lombok

@Getter, @Builder,  
@RequiredArgsConstructor.



### Jackson

@JsonInclude,  
@JsonProperty.



### Avoid @Data by Default

Also sets  
equals/hashCode/toString.

- DTO PROMPT: "Create a DTO for creating a customer with validation annotations for fullName and email." DIFFERENCES TO REVIEW: a DTO carries no business logic — keep it focused on only the fields the API needs, map to/from Customer explicitly, and keep request/response DTOs separate from the entity.

**KEY TAKEAWAY** Copilot drafts a first pass of a class or DTO — review, test, and align it with your standards before production use.



# TRY IT YOURSELF — EXERCISE 2: CUSTOMER SKETCH FOR AMINA

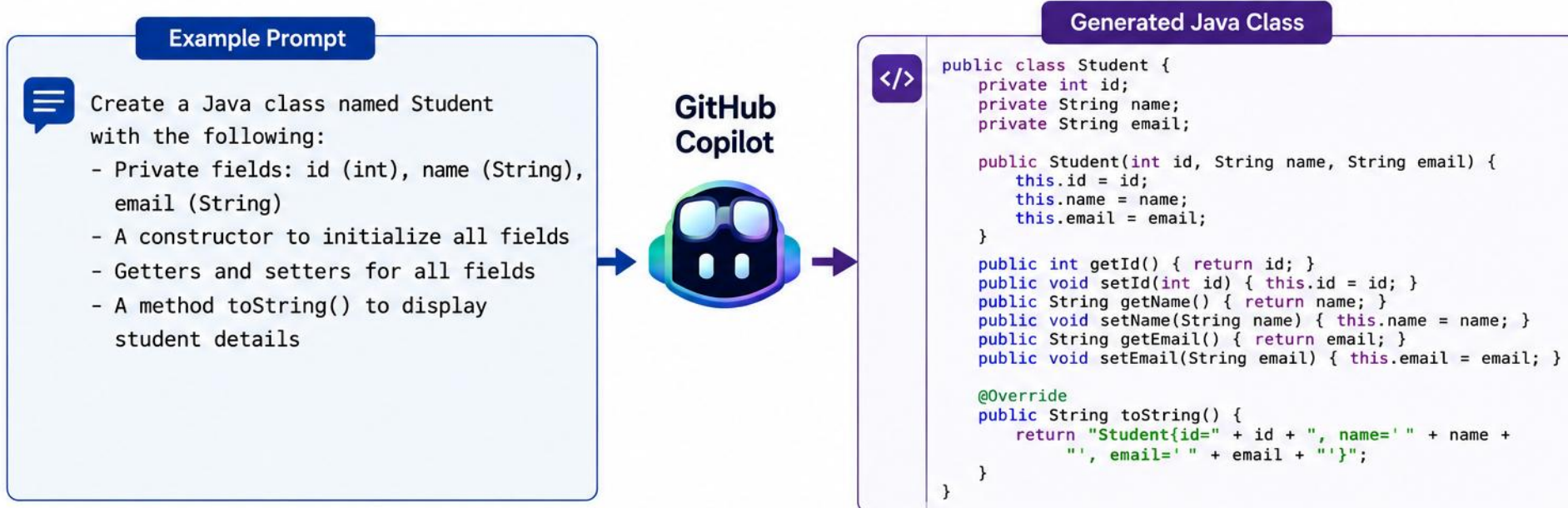
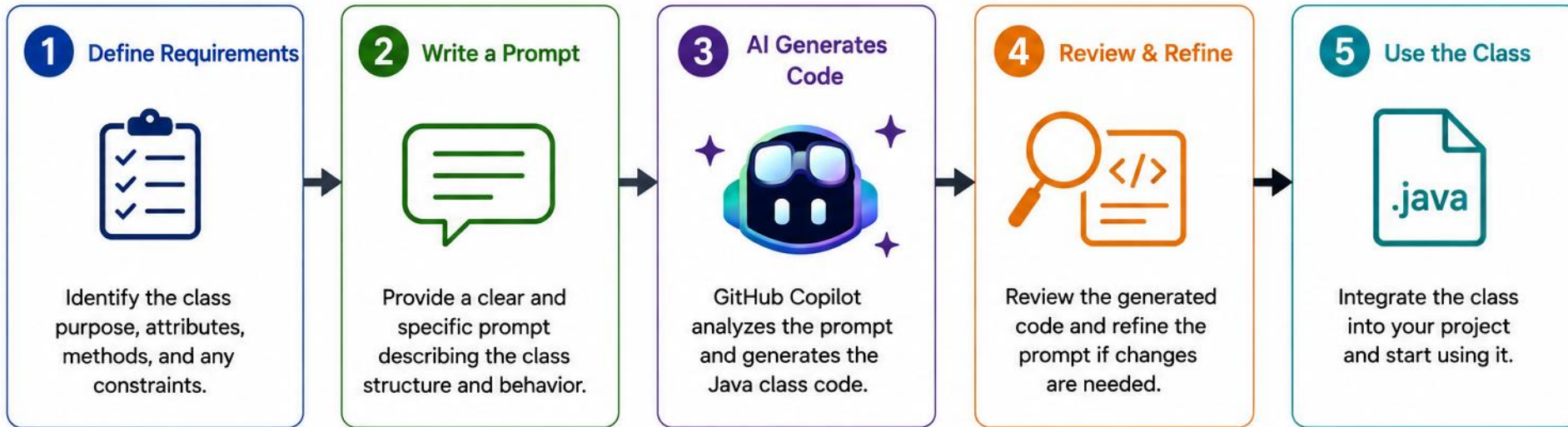
*Reinforces: the Customer class fields Copilot just generated for you.*

| STEP             | WHAT TO DO                                                                 |
|------------------|----------------------------------------------------------------------------|
| Goal             | Hand-sketch fields for CUS-1001 before asking Copilot to generate any code |
| File             | customer-sketch-notes.md (in examples/module-10-exercises/notes/)          |
| Amina row        | CUS-1001, Amina Khan, status ACTIVE — customerId, fullName, status         |
| Ravi row         | CUS-1002, Ravi Singh, status PROSPECT — same shape, different values       |
| Correlation note | lab-request-001 belongs in logs/headers later, not as a Customer field     |
| Reference        | exercises/exercise-02-customer-sketch.md                                   |

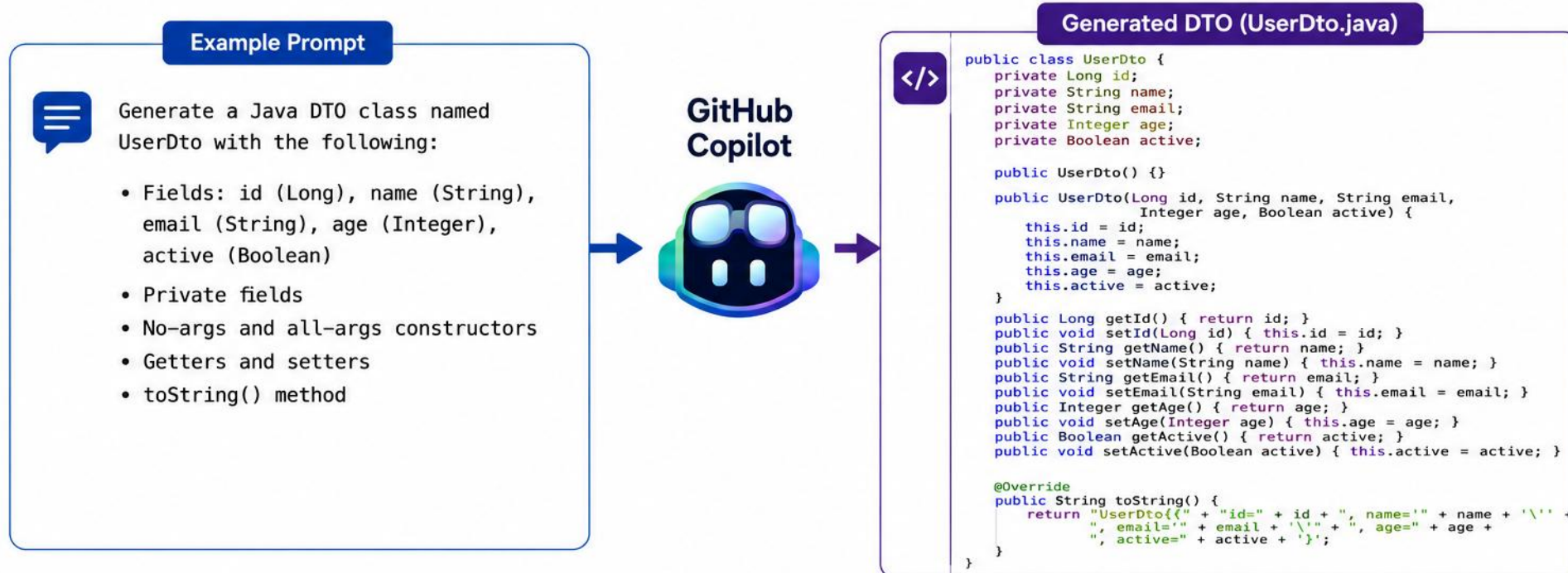
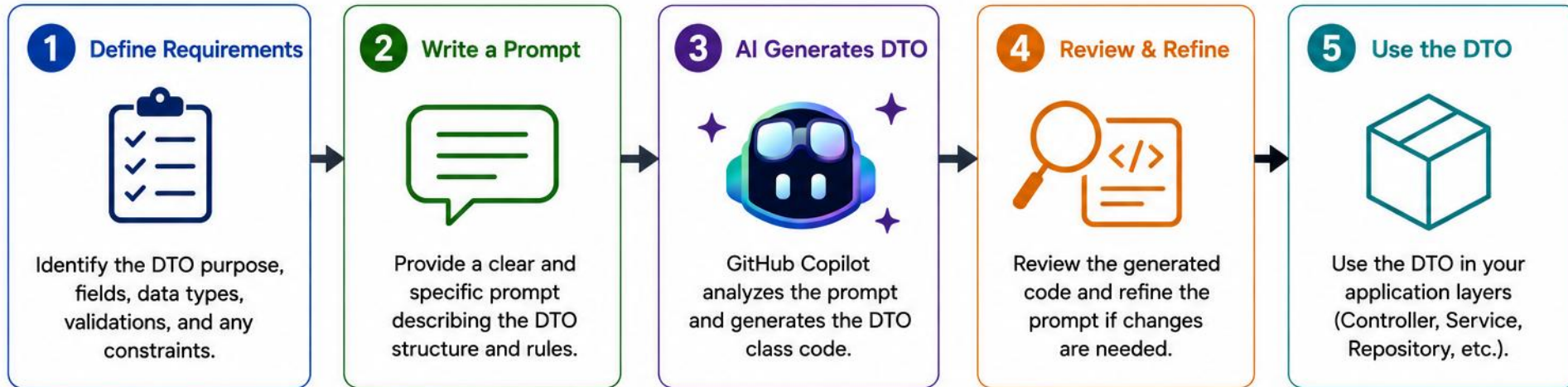
```
$ cat notes/customer-sketch-notes.md
customerId    fullName      status
CUS-1001      Amina Khan    ACTIVE
CUS-1002      Ravi Singh    PROSPECT
lab-request-001 // logs/headers only – not a Customer field
```

**TRY IT NOW** Complete Exercise 2 before continuing — see exercises/exercise-02-customer-sketch.md.

# Generating Java Classes



# Generating DTOs



# GENERATING SERVICE LOGIC

*Service classes hold business logic — validation, search/filter/update rules, and error handling — that Copilot can draft and you refine.*

## HOW TO GENERATE A SERVICE

- Describe the service's responsibilities, methods, and key dependencies.
- Begin typing the class name or key methods — Copilot suggests the rest.
- Focus on business logic — Copilot fills in method bodies and structure.
- Review generated code and refine error handling and validations.

## INDUSTRY PREVIEW — SPRING BOOT ANNOTATIONS

| ANNOTATION               | PURPOSE                              |
|--------------------------|--------------------------------------|
| @Service                 | Marks a business-layer component.    |
| @Repository              | Marks a data-access layer component. |
| @Transactional           | Runs the method in a transaction.    |
| @Valid                   | Validates method parameters.         |
| @RequiredArgsConstructor | Generates constructor (Lombok).      |

# SERVICE LOGIC IN PRACTICE

*See how the CustomerService for Northstar CRM puts these prompts and annotations to work in plain Java.*

## CUSTOMERSERVICE — PLAIN JAVA



### Service Prompt

Add, findById, search, updateStatus.



### Business Rules & Errors

Validate input; throw clear exceptions.



### Testability

Inject the list; unit test with JUnit.

**KEY TAKEAWAY** Copilot can draft service logic fast — you still own validation, error handling, and tests before it ships.



# TRY IT YOURSELF — EXERCISE 3: PHANTOM ANNOTATION HUNT

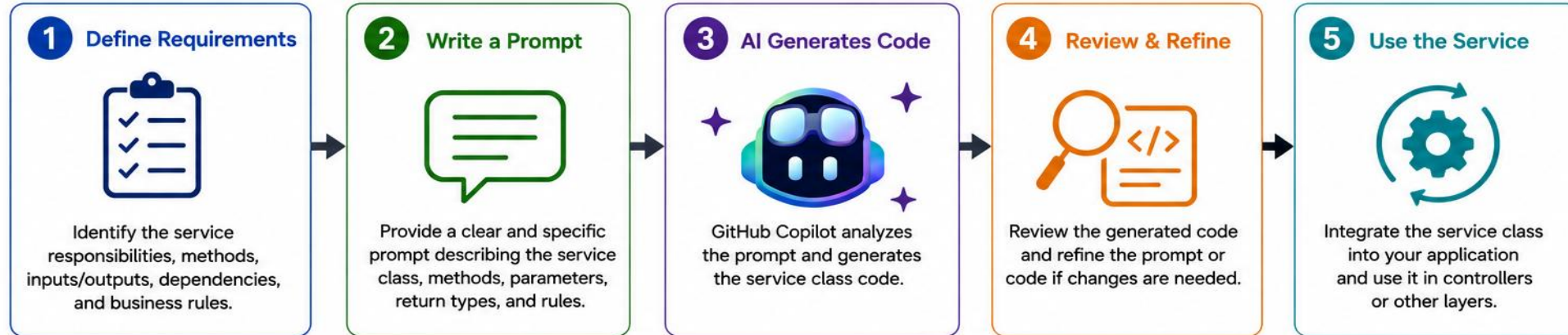
*Reinforces: the Spring Boot annotations you just saw marked Industry Preview.*

| STEP          | WHAT TO DO                                                                    |
|---------------|-------------------------------------------------------------------------------|
| Goal          | Flag Copilot-style annotations that do not belong in a plain Java prep sketch |
| File          | phantom-annotation-notes.md (in examples/module-10-exercises/notes/)          |
| Defer         | @Entity/@Table (JPA), @Service/@Autowired (Spring) — not Lab 10 scope yet     |
| Reject rule   | Reject any import you cannot name from JDK 21 or an agreed Maven dependency   |
| Fixture check | CUS-1002 Ravi hard-coded as ACTIVE is wrong — correct status is PROSPECT      |
| Reference     | exercises/exercise-03-phantom-annotation-hunt.md                              |

```
$ cat notes/phantom-annotation-notes.md
@Entity / @Table // JPA only – defer, not Lab 10 scope
@Service / @Autowired // Spring – defer, hosting labs later
@NotNull (Jakarta) // name it, never invent the import
public record Customer(...) // OK on JDK 21
```

**TRY IT NOW** Complete Exercise 3 before continuing — see exercises/exercise-03-phantom-annotation-hunt.md.

# Generating Service Classes



## Example Prompt



Generate a Java Spring service class named UserService with the following:

- Use @Service annotation
- Dependency: UserRepository userRepository
- Methods:
  - UserDto getUserById(Long id)
  - List<UserDto> getAllUsers()
  - UserDto createUser(UserDto userDto)
  - UserDto updateUser(Long id, UserDto userDto)
  - void deleteUser(Long id)
- Use User entity and UserDto
- Add basic business logic and mapping between Entity and DTO

GitHub Copilot



## Generated Service Class (UserService.java)

```

</>
@Service
@RequiredArgsConstructor
public class UserService {
    private final UserRepository userRepository;

    public UserDto getUserById(Long id) {
        User user = userRepository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("User not found"));
        return mapToDto(user);
    }

    public List<UserDto> getAllUsers() {
        return userRepository.findAll().stream()
            .map(this::mapToDto)
            .toList();
    }

    public UserDto createUser(UserDto userDto) {
        User user = mapToEntity(userDto);
        User saved = userRepository.save(user);
        return mapToDto(saved);
    }

    public UserDto updateUser(Long id, UserDto userDto) {
        User user = userRepository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("User not found"));
        user.setName(userDto.getName());
        user.setEmail(userDto.getEmail());
        user.setActive(userDto.getActive());
        return mapToDto(userRepository.save(user));
    }

    public void deleteUser(Long id) {
        userRepository.deleteById(id);
    }

    private UserDto mapToDto(User user) {
        return new UserDto(user.getId(), user.getName(), user.getEmail(), user.getActive());
    }

    private User mapToEntity(UserDto dto) {
        return new User(null, dto.getName(), dto.getEmail(), dto.getActive());
    }
}
  
```

# GENERATING DOCUMENTATION

*Clear documentation makes your code easy to understand, maintain, and use — Copilot can generate it in seconds.*

## **REVIEW CHECKLIST — BEFORE YOU SHIP DOCUMENTATION**

- Does the documentation match the code's actual behavior?
- Are the parameter and return descriptions accurate?
- Are exceptions documented?
- Are setup commands verified?
- Does it avoid stating unsupported guarantees?

# DOCUMENTATION TOOLS AND TIPS

*Copilot can generate several kinds of documentation in seconds — use these tips to keep it accurate.*

## WHAT COPILOT CAN GENERATE



### JavaDoc

Method and class-level docs.



### README

Project overview and setup guides.



### API Docs

REST endpoint documentation.



### Inline Comments

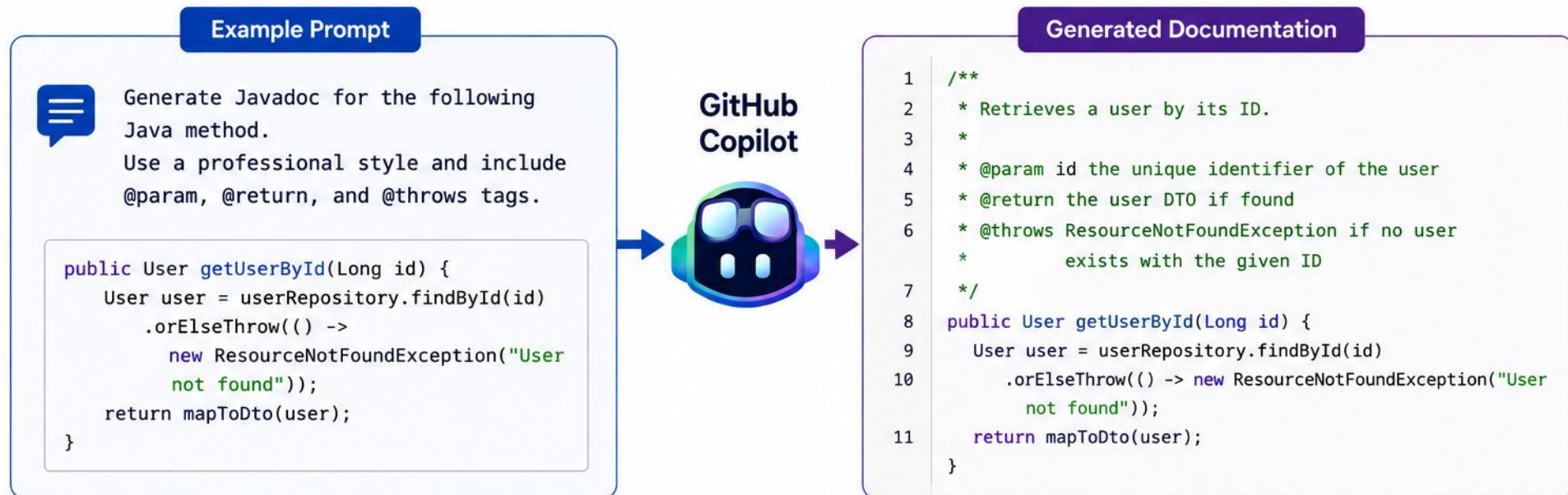
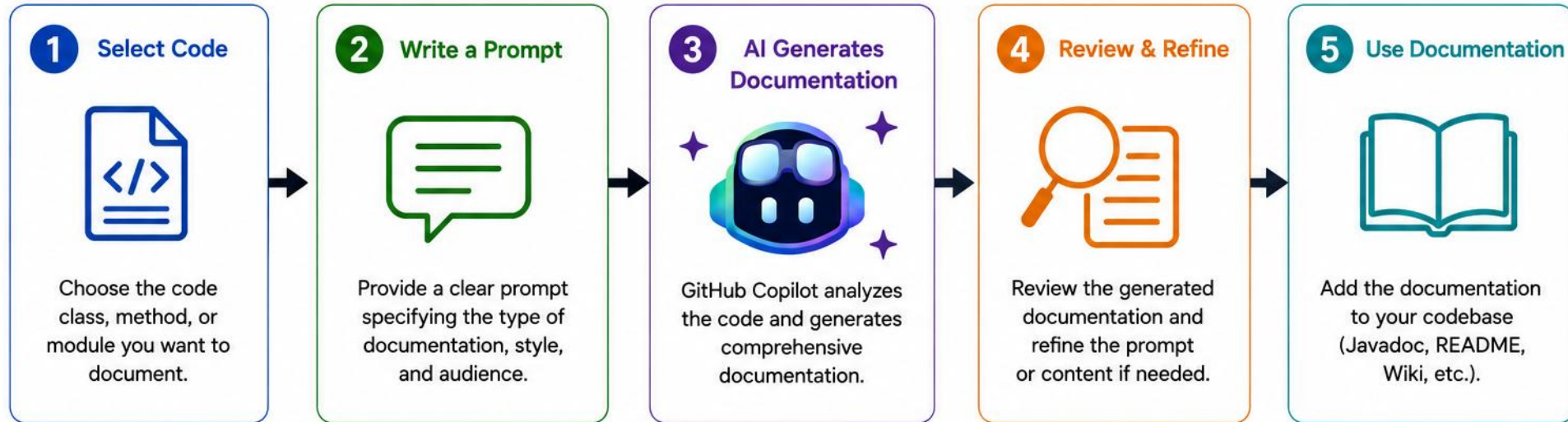
Explain non-obvious logic.

## TIPS FOR BETTER DOCUMENTATION

- Be specific in your prompt (e.g. "Generate JavaDoc for this method"). Keep docs in sync with code changes, and follow your project's documentation style and conventions.

**KEY TAKEAWAY** Good documentation is as important as good code — with Copilot, you can generate high-quality docs quickly.

# Generating Documentation

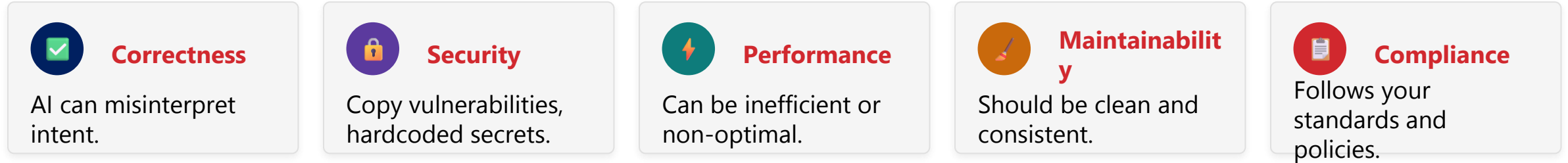




# REVIEWING AI-GENERATED CODE

*AI-generated code can save time, but it's your responsibility to review it for correctness, security, and maintainability. Do: use meaningful names and provide clear, complete prompts. Don't: accept a suggestion blindly or copy code you don't understand.*

## WHY YOU MUST REVIEW



## REVIEW WORKFLOW



# REVIEW IN PRACTICE: SQL INJECTION

*Applying the review workflow to a real suggestion: an unreviewed query is vulnerable until you refactor it.*

## EXAMPLE: PREPAREDSTATEMENT VS STRING CONCATENATION

```
// UNREVIEWED – SQL injection risk
String sql = "SELECT * FROM user WHERE id=" + id;
// REVIEWED & IMPROVED – parameterized query
String sql = "SELECT * FROM users WHERE id = ?";
try (PreparedStatement stmt = connection.prepareStatement(sql)) {
    stmt.setString(1, id);
}
```

**KEY TAKEAWAY** Never blindly trust AI-generated code. Always understand before you accept — review smart, deploy with confidence.

# TRY IT YOURSELF — EXERCISE 4: FILL REVIEW-LOG TODOS (STARTER)

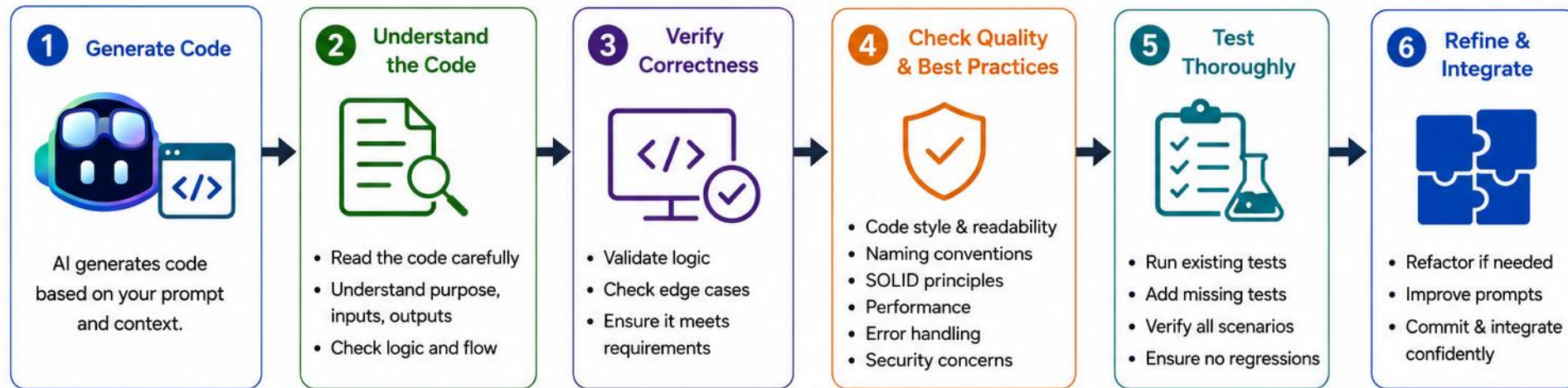
*Reinforces: the six-step review workflow — Get, Understand, Review, Refactor, Test, Validate.*

| STEP       | WHAT TO DO                                                                                   |
|------------|----------------------------------------------------------------------------------------------|
| Goal       | FILL-IN-THE-BLANK STARTER — complete a Copilot review log before the timed lab               |
| File       | lab10-review-log-todos.md (in examples/module-10-exercises/notes/)                           |
| Fill in    | 6 blanks: prompt strength, phantom annotation, Amina/Ravi fixtures, JDK/Maven note, decision |
| Rule       | Replace every single ____ with a concrete value — never leave a blank empty                  |
| Self-check | Confirm Ravi is PROSPECT and Amina is ACTIVE before you claim prep is done                   |
| Reference  | exercises/exercise-04-fill-review-log-todos.md                                               |

Prompt strength: \_\_\_\_  
Phantom annotation found? \_\_\_\_ // yes/no + name  
Fixture check – Amina: \_\_\_\_ Ravi: \_\_\_\_  
JDK/Maven note: \_\_\_\_  
Accept / Reject / Edit: \_\_\_\_

**TRY IT NOW** Complete Exercise 4 before continuing — see exercises/exercise-04-fill-review-log-todos.md (starter).

# Reviewing AI-Generated Code



## Code Review Checklist

|  <b>FUNCTIONALITY</b>                                                                                                                  |  <b>CODE QUALITY</b>                                                                                                                |  <b>PERFORMANCE</b>                                                         |  <b>SECURITY</b>                                                                            |  <b>TESTING</b>                                                                                                               |  <b>DOCUMENTATION</b>                                                                           |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>✓ Does the code meet the requirement?</li><li>✓ Are inputs/outputs handled correctly?</li><li>✓ Are edge cases considered?</li><li>✓ Is the logic correct and complete?</li></ul> | <ul style="list-style-type: none"><li>✓ Is the code readable and well-structured?</li><li>✓ Are names meaningful and consistent?</li><li>✓ Is duplication avoided?</li><li>✓ Is the code easy to maintain?</li></ul> | <ul style="list-style-type: none"><li>✓ Is the algorithm efficient?</li><li>✓ Are loops/queries optimized?</li><li>✓ Is resource usage appropriate?</li></ul> | <ul style="list-style-type: none"><li>✓ Are inputs validated and sanitized?</li><li>✓ Is sensitive data protected?</li><li>✓ Are there any security vulnerabilities?</li></ul> | <ul style="list-style-type: none"><li>✓ Are unit tests present and relevant?</li><li>✓ Do tests cover normal, boundary &amp; error cases?</li><li>✓ Do all tests pass?</li><li>✓ Any new tests needed?</li></ul> | <ul style="list-style-type: none"><li>✓ Is the code documented appropriately?</li><li>✓ Are comments clear and helpful?</li><li>✓ Is the intent clear to future readers?</li></ul> |

## DECISION



# RISKS OF AI-ASSISTED DEVELOPMENT (1 OF 2)

*AI tools boost productivity but introduce new risks — understanding them helps you use AI responsibly.*

## **Incorrect or Inaccurate Code**

- Syntactically correct but logically wrong or incomplete.

## **Security Vulnerabilities**

- Insecure patterns or exposed sensitive data.

## **License & IP Risks**

- May reproduce copyrighted code snippets.

## **Over-Reliance on AI**

- Reduced critical thinking and learning.



# RISKS OF AI-ASSISTED DEVELOPMENT (2 OF 2)

*Four more risks to watch for, plus practical controls for the License & IP Risk category.*



## Lack of Context Awareness

- Doesn't understand full system or business rules.



## Data Privacy Concerns

- Prompts may leak sensitive or proprietary data.



## Bias & Ethical Concerns

- May reflect training-data bias, non-inclusive outcomes.



## Poor Code Quality

- Can be verbose or misaligned with project standards.

### PRACTICAL CONTROLS FOR LICENSE & IP RISK (NOT LEGAL ADVICE)

- Review suggestions for unexpectedly long or distinctive code, and follow your organization's IP policy.
- Use code-referencing/filtering features where available, and validate third-party license compatibility.
- Escalate uncertain cases to legal/compliance.

**KEY TAKEAWAY** AI is a powerful assistant, not a replacement for engineering judgment. Review, verify, validate — you own the outcome.

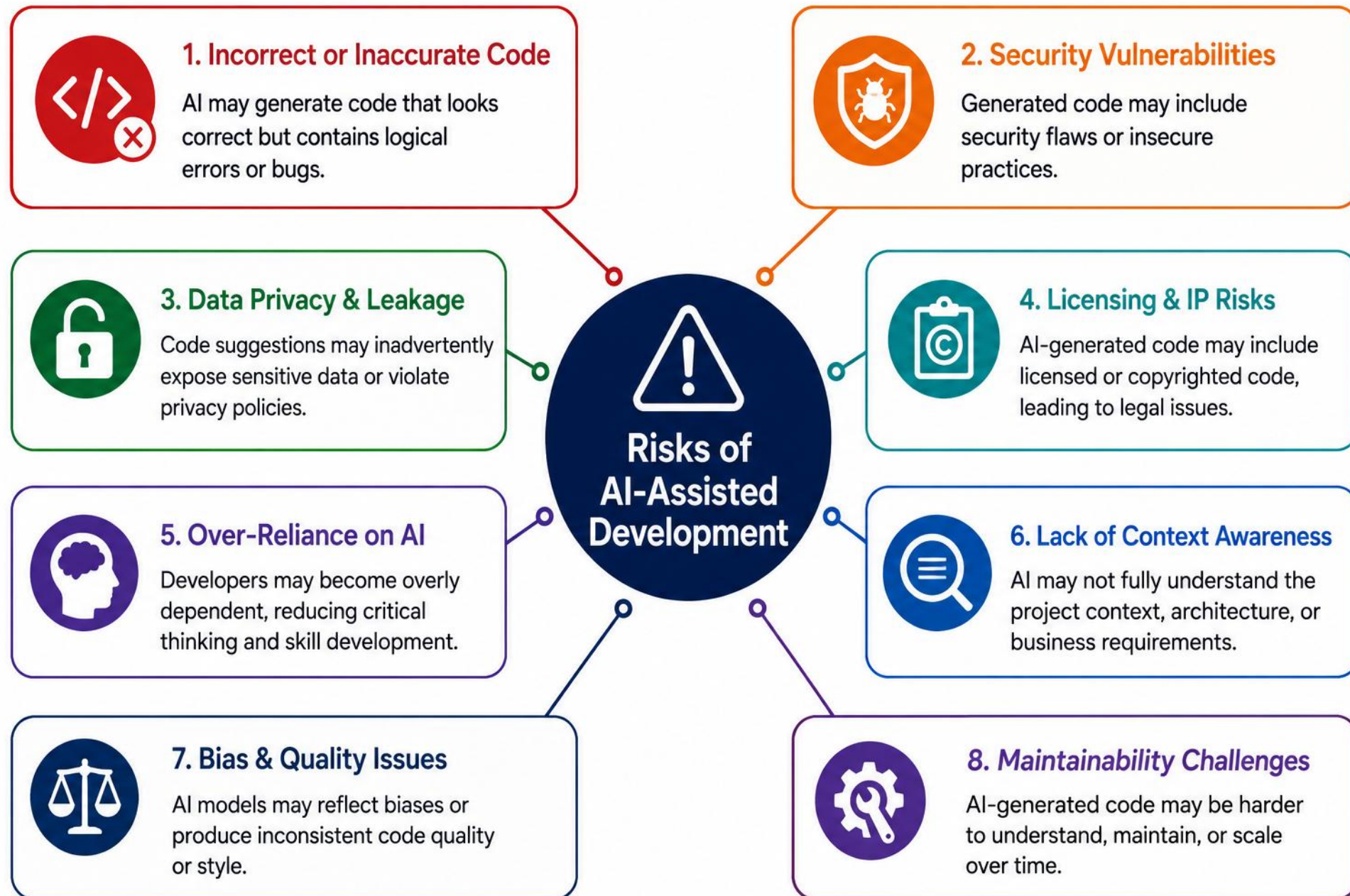
# TRY IT YOURSELF — EXERCISE 5: JDK 21 / MAVEN HABIT

*Reinforces: Lack of Context Awareness — Copilot doesn't know your real toolchain.*

| STEP      | WHAT TO DO                                                                  |
|-----------|-----------------------------------------------------------------------------|
| Goal      | Record the exact version checks you will run before Lab 10 coding           |
| File      | jdk-maven-checklist.md (in examples/module-10-exercises/notes/)             |
| Commands  | java -version (expect 21.x) and mvn -version (expect Java home = 21)        |
| PATH trap | Wrong JDK first on Windows PATH — reorder PATH before the lab, don't assume |
| Workspace | Bootcamp workspace + notes\ for this module's prep files                    |
| Reference | exercises/exercise-05-jdk-maven-habit.md                                    |

```
$ java -version
openjdk version "21...  // must read 21.x, not 17 or 11
$ mvn -version
Java version: 21..., Java home: ...jdk-21
# Do not run full lab Maven goals until the timed Lab 10 session
```

**TRY IT NOW** Complete Exercise 5 before continuing — see exercises/exercise-05-jdk-maven-habit.md.



# ENTERPRISE USE CASES

*GitHub Copilot supports Java teams across the development lifecycle — from everyday scaffolding to enterprise-scale delivery.*



## Refactoring Legacy Java

Modernize old modules and update patterns.



## Migration Assistance

Help port code between frameworks or versions.



## API & Library Exploration

Discover unfamiliar APIs and libraries faster.



## Secure Code Review Support

Detect and fix vulnerable patterns.



## Test Automation

Unit, integration, and end-to-end tests.



## Explaining Unfamiliar Code

Faster ramp-up on unfamiliar codebases.



## Documentation

README, API docs, JavaDoc at scale.



## Boilerplate & Scaffolding

Generate classes, DTOs, and config fast.

# BUSINESS IMPACT AT ENTERPRISE SCALE

*These use cases translate into measurable outcomes for engineering teams, across every regulated industry.*

**Faster**  
Delivery

**Higher**  
Quality

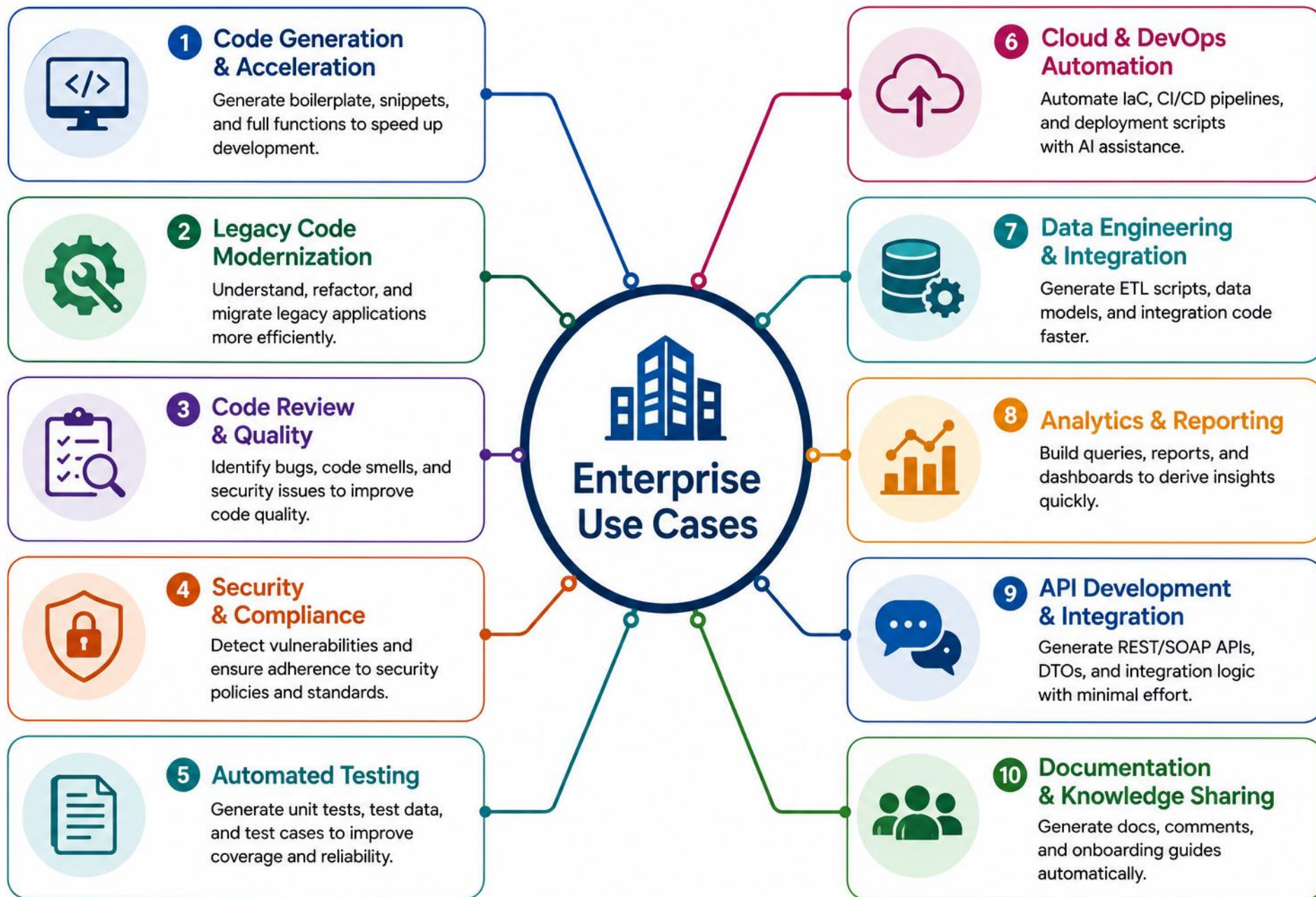
**Lower**  
Risk

**By Design**  
Privacy

- Industries: Financial Services, Healthcare, Retail, Manufacturing, Public Sector. Enterprise controls may include organization policies, access management, audit capabilities, content controls, and data-governance options — confirm current capabilities for your GitHub plan.

**KEY TAKEAWAY** Copilot is more than a coding assistant — it's a force multiplier for engineering teams, used responsibly.





# TRY IT YOURSELF — EXERCISE 6: LAB 10 PREP CHECKLIST

*Reinforces: Exercises 1–5 together — confirm you’re ready before the timed lab.*

| STEP      | WHAT TO DO                                                                           |
|-----------|--------------------------------------------------------------------------------------|
| Goal      | Confirm prep readiness without completing Lab 10 itself                              |
| File      | lab10-prep-checklist.md (in examples/module-10-exercises/notes/)                     |
| Lab asks  | Three things from the Lab 10 header: prompts, review log, generated sketch           |
| Fixtures  | CUS-1001 Amina ACTIVE, CUS-1002 Ravi PROSPECT, correlation lab-request-001           |
| Pass rule | Pass if prompts + phantom checklist + review TODOs exist; else revisit Exercises 1–4 |
| Reference | exercises/exercise-06-lab10-prep-checklist.md                                        |

```
$ cat notes/lab10-prep-checklist.md
[ ] Weak vs strong prompts documented
[ ] Phantom annotations flagged, reject rule written
[ ] Customer sketch: CUS-1001 Amina ACTIVE, CUS-1002 Ravi PROSPECT
[ ] Review-log TODOs filled; java/mvn confirmed on JDK 21
```

**TRY IT NOW** Complete Exercise 6 before continuing — see exercises/exercise-06-lab10-prep-checklist.md.

# LAB OVERVIEW – AI-ASSISTED CODE GENERATION

*Use GitHub Copilot to scaffold the Northstar CRM customer domain — with mandatory human review of every suggestion.*

## LAB SCENARIO

- Extend the Northstar CRM customer-service Maven project (from Lab 9) with a Customer domain.
- Customer has customerId, fullName, email, phone, status, and createdAt.
- Plain Java 21 and Maven only — no Spring, no JPA, no REST yet.
- Required workflow: Prompt → Review → Refine → Test.

## REQUIRED WORKFLOW



## LAB ENVIRONMENT

| ITEM            | VALUE                     |
|-----------------|---------------------------|
| Language        | Java 21                   |
| Framework       | None (plain Java + Maven) |
| Data Store      | In-memory List<Customer>  |
| Version Control | Git & GitHub              |

**KEY TAKEAWAY** Generate, review, refine, and document — use Copilot to accelerate, not to replace your thinking.

# LAB TASKS AND DELIVERABLES (1 OF 2)

*Complete the tasks, review and refine the AI-generated code, and deliver a working, reviewed customer domain.*

| # | TASK                   | DETAILS                                                                      | WEIGHT |
|---|------------------------|------------------------------------------------------------------------------|--------|
| 1 | Setup & Sign-In        | Sign in to Copilot; copy lab9-crm to lab10-crm; verify Copilot status Ready. | 10%    |
| 2 | Weak vs Strong Prompts | Compare vague vs. specific prompts; log entries lab10-001 / lab10-002.       | 15%    |
| 3 | Entity, Enum & Service | Scaffold CustomerStatus, Customer, and CustomerService via Copilot Chat.     | 25%    |

# LAB TASKS AND DELIVERABLES (2 OF 2)

*Tasks 4-6, plus the deliverables checklist for what you submit.*

| # | TASK                       | DETAILS                                                        | WEIGHT |
|---|----------------------------|----------------------------------------------------------------|--------|
| 4 | Main Harness               | Prove behavior for CUS-1001 / CUS-1002; mvn clean compile.     | 15%    |
| 5 | Human Review & Experiments | Checklist lab10-003; 3+ documented failure experiments.        | 15%    |
| 6 | Risk Notes & Docs          | ai-review-notes.md lab10-004; security and production answers. | 20%    |

## DELIVERABLES

- Customer entity, CustomerStatus enum, and CustomerService compiling clean; a Main harness proving CUS-1001 / CUS-1002; ai-review-notes.md with entries lab10-001–004; no secrets or target/ committed.
- ☐ **Required in ai-review-notes.md:** one accepted suggestion, one modified suggestion, and one rejected suggestion, with the reason for each decision and which tests validated the final code.

**KEY TAKEAWAY** AI tools can accelerate development, but your judgment, review, and testing deliver quality.



# COPILOT DEFINITION OF DONE

*Before you call a Copilot-assisted change finished, check it against this practical checklist.*

- 1 Prompt includes the goal, context, constraints, and acceptance criteria.
- 2 The suggestion is understood — not just accepted.
- 3 Code follows your project's standards and conventions.
- 4 Tests cover normal cases and edge cases.
- 5 Security and privacy have been reviewed.
- 6 Dependencies and APIs used are verified as real.
- 7 Documentation matches the code's real behavior.
- 8 A human reviewer approves the final change.

**KEY TAKEAWAY** AI suggests, you decide. Review every suggestion, understand before you accept, and build with quality and standards.

# KNOWLEDGE CHECK (1 OF 2)

*Test your understanding of GitHub Copilot concepts, best practices, and AI-assisted development.*

## 1. What's the primary benefit of GitHub Copilot?

- A. It replaces developers
- B. It automatically tests the application
- C. It writes code faster and reduces busywork**
- D. It deploys the application to production

## 3. What's a best practice for writing prompts?

- A. Be vague to get creative results
- B. Provide clear context and specific requirements**
- C. Ask Copilot to write the entire app in one prompt
- D. Avoid describing the expected output

## 2. What should you do right after Copilot generates code?

- A. Review and understand the generated code**
- B. Run it in production immediately
- C. Delete it and start over
- D. Share it on social media

## 4. What should you always verify in AI-generated code?

- A. Variable and method names
- B. Syntax and code style
- C. Business logic correctness and security
- D. All of the above**

**KEY TAKEAWAY** Copilot helps developers write code faster — but correctness, security, and style all still need human review.

# KNOWLEDGE CHECK (2 OF 2)

*Four more questions, plus the full answer key.*

## 5. What should you do after Copilot generates code?

- A. Accept it as-is without any review
- B. Run it in production to test
- C. Review, understand, test, and refine it**
- D. Share it with your team immediately

## 7. What's a recommended enterprise Copilot practice?

- A. Share all prompts and code publicly
- B. Follow organization coding standards**
- C. Disable code reviews
- D. Ignore security scanning

### ANSWER KEY

• 1-C 2-A 3-B 4-D 5-C 6-B 7-B 8-B

## 6. Which layer holds business logic in a REST API?

- A. Controller Layer
- B. Service Layer**
- C. Repository Layer
- D. DTO Layer

## 8. What is the main goal of refining AI-generated code?

- A. To make it longer
- B. To improve readability, performance, and maintainability**
- C. To change variable names only
- D. To avoid writing any code manually

**KEY TAKEAWAY** Use Copilot intentionally. Review critically. Build securely. Document clearly. Deliver high-quality software.

## MODULE 10 COMPLETE

# GitHub Copilot Fundamentals for Java Developers

You can now use GitHub Copilot to generate, review, and refine code responsibly — a force multiplier, not a replacement for judgment.